AgentLedger: A Causal Ledger Architecture for Auditable AI Agent Actions

[Donald Fernandes, Lakshana Pant] |

Independent Research | March 2026

Abstract

AI agents are being deployed in consequential settings faster than anyone is building the tools to govern them. When an agent goes wrong today - sends an email it shouldn't have, deletes the wrong records, fires an unauthorized API call - the post-incident artifacts tell you *what* happened, never *why*. The reasoning chain that produced the action is simply gone. This paper proposes AgentLedger: an architecture that records every consequential agent action as a cryptographically linked block carrying a versioned snapshot of the agent's full environment, a causal graph of the reasoning that led to the action, and a deterministic attribution of that action to specific environmental changes. The record is tamper-evident, independently verifiable, and built to satisfy the accountability requirements of regulated industries. AgentLedger draws on two well-proven cryptographic systems - Google's Certificate Transparency log and Ethereum's state root mechanism - and applies their integrity primitives to the agent accountability problem. What follows is the architecture, a concrete failure scenario showing it in action, a mapping to current regulatory requirements, and an honest account of what's still unsolved.

---

1. The Problem

When an AI agent fails in production, the debugging experience is, frankly, unpleasant. You have token logs if you remembered to save them, tool call records if your framework happens to emit them, and application logs if their timestamps happen to line up with what the agent did. These artifacts are scattered across different systems and share one fundamental limitation: they record *what* the agent did, not *why*.

That distinction matters differently depending on who's asking.

For **engineers**, agents simply cannot be debugged the way deterministic software can. A stack trace gives you the call path from input to failure - there is no equivalent for "the model misread the user's intent at step 3 because of ambiguous context injected at step 1." The connection between environment state and action is invisible to existing tooling, which means debugging an agent failure is mostly informed guesswork. Good engineers hate that.

For **regulated industries**, the gap becomes a compliance liability. Finance, healthcare, and legal deployments operate under frameworks - GDPR Article 22, SOX, the EU AI Act - that require demonstrable accountability for automated decisions. "The LLM decided" is not an auditable answer. Regulators want evidence of what inputs were present, what constraints were in effect, and how the decision was reached. No current system produces that evidence in a form that can be independently verified.

For **incident response**, the gap prevents replay. To determine whether a failure stemmed from a prompt issue, a tool boundary violation, a context contamination event, or a model capability gap, you need to reconstruct the full environment state at the time of failure. Without versioned, committed environment snapshots, that reconstruction is either impossible or unverifiable - and "probably the prompt" is not a root cause.

This gap is structural, not incidental. Agent frameworks are designed to maximize capability. Logging systems record events. Neither is designed to produce auditable causal records, and the result is a class of failures that are visible after the fact but cannot be rigorously attributed.

---

2. What AgentLedger Proposes

AgentLedger is a causal ledger for AI agent actions. The central claim is straightforward: every consequential decision an agent makes should be recorded as a cryptographically linked block - analogous to a blockchain block, but without any distributed network - containing three things:

1. **A versioned snapshot of everything that could have influenced the agent's behavior** at the moment of the action. This includes the system prompt, the model and its parameters, available tools, the contents of the context window, RAG retrieval results, and the state of the agent's memory. We call this the *Change Envelope*.
2. **A causal graph** - a map of how the agent's reasoning steps depended on each other and on the environment state. This enables attribution: given a failure, which part of the environment was actually on the causal path to the wrong action?
3. **A deterministic ranking** of which environmental changes most likely caused the failure. Deterministic here means the same inputs always produce the same verdict. The language model plays no role in this ranking - it only generates a human-readable explanation after the ranking is computed. An LLM ranking why another LLM failed is circular and ungovernable. That distinction is worth being explicit about.

Together, these three elements constitute a *causal record*. It is tamper-evident - stored in an append-only, cryptographically chained log where altering any entry breaks the chain. It is independently verifiable - an auditor can check the integrity of the record without access to any internal systems. And it is selectively disclosable - individual components of the environment snapshot can be proven without revealing full context, which matters in regulated settings where data exposure is tightly constrained.

---

3. The Cryptographic Foundations

AgentLedger's integrity properties are not novel cryptography. They are established mechanisms applied to a new domain, and the analogy to prior systems is stronger than it might first appear.

Google Certificate Transparency (CT), introduced in 2013, solved a trust problem in TLS certificate issuance: how do you know a certificate record wasn't created or modified after the fact, without having to trust the issuing authority? CT's answer is a public, append-only log backed by a Merkle tree. Every new entry becomes a leaf. The tree's root hash is periodically signed and published. Anyone can verify that a specific entry is in the log and that the log has only grown - never been altered. Critically, these proofs are self-contained: you don't need access to internal CA systems to verify an audit trail.

AgentLedger needs exactly this property for agent actions. An auditor must be able to verify that a causal record was not constructed retroactively, without requiring access to internal agent infrastructure. CT's append-only Merkle log is the direct model for AgentLedger's ledger layer.

Ethereum's state root mechanism solves a different problem: how do you produce a compact, verifiable fingerprint of a large and continuously changing world state? Each Ethereum block contains a state root - a single hash that cryptographically commits to the entire system state at that moment. Any change anywhere produces a different root hash. The difference between two state roots tells you exactly what changed between them.

AgentLedger's Change Envelope applies this directly to agent environments. The envelope root hash commits to everything that could influence agent behavior - prompt, model, tools, context, memory - at a specific point in time. The diff between two envelope hashes tells you precisely which components changed between two agent steps. This is what makes causal attribution tractable: not just that the environment changed, but exactly which pieces changed and when.

What AgentLedger does *not* adopt is distributed consensus. No blockchain network, no proof-of-work, no decentralized validators. The accountability problem here is organizational - a single company governing its own agent deployments - and the architecture is scoped accordingly. The value is in the data structures and integrity primitives, not the network topology.

---

4. A Concrete Example

Consider a customer success agent deployed at a SaaS company. It has access to the customer database, the email system, and a knowledge base retrieved via RAG. Its job is to handle escalated support tickets.

On a Tuesday afternoon, the agent sends a renewal pricing email to a customer whose account is flagged as churned and in active legal dispute. This is precisely the kind of action that should never happen. It did.

What the current post-incident record looks like:

- Token logs showing the agent generated an email
- A tool call record showing `send_email(to=customer@example.com, ...)`
- An application log entry timestamped 2:47 PM

These tell you what happened. They don't tell you why. Was it a prompt that failed to account for the legal-dispute flag? A RAG retrieval that returned a stale customer segment? A memory item from a prior session that misclassified the account? You don't know. You're reconstructing a crime scene with no forensics.

What AgentLedger would capture:

At session start, AgentLedger captures a baseline Change Envelope - a cryptographic snapshot of the system prompt hash, model version and parameters, tool schemas, and critically, the RAG retrieval results: which query was run, which chunks were returned, and their hashes at retrieval time.

At the moment `send_email` is called, AgentLedger captures a second envelope and computes the delta. In this case, the delta shows that the context state changed at step 2: a retrieval call returned a cached customer segment record that hadn't been updated to reflect the legal-dispute flag added three days earlier.

AgentLedger then computes which reasoning steps were on the causal path to `send_email`, and ranks the RAG context change as the top-ranked cause - it's on the causal path, it's the closest environmental change to the failure point, and it uniquely explains the trajectory to the wrong action.

The resulting causal record reads: *at step 2, the context state changed due to a stale RAG retrieval. This change was on the causal path to the send_email action at step 4. The retrieved chunk did not contain the legal-dispute flag present in the live database. This is the top-ranked causal candidate.*

That record is append-only, hash-chained, and independently verifiable. A compliance officer, an engineer, or a regulator can examine it and understand exactly what happened - not "the LLM decided," but a structured, replayable causal narrative that points to a specific, fixable engineering failure.

---

5. The Environment State Model

The Change Envelope captures six components of agent environment state. Each is stored as a cryptographic hash, not raw content. Full content lives separately in an encrypted store, keyed by hash, and is only accessed when an authorized audit requires it. This keeps the ledger compact while preserving the ability to verify and replay.

Agent Identity - which agent, which version, which deployment, which session. Stable within a session; meaningful when it changes across them.

Model State - which model, which version, what inference parameters (temperature, sampling configuration). A model version change between sessions is a real environmental delta, not a bookkeeping detail.

Instruction State - the active system prompt and any instruction modifications applied during the session. Prompt changes are among the most common sources of behavioral drift and are, remarkably, frequently unversioned in production systems.

Tool State - the set of tools available and their schemas at this moment. A schema change - even a minor one - can change how the agent constructs arguments and what actions it considers available. This is underappreciated.

Context State - the contents of the context window and, critically, the results of any RAG retrieval: what query was run, what chunks were returned, and what their hashes were at retrieval time. RAG retrieval is non-deterministic - the same query can return different chunks as the vector store evolves. Most agent systems don't version retrieval results at all. Making this explicit and hashable is one of AgentLedger's primary contributions.

Memory State - the persistent memory available to the agent, with provenance: which agent wrote each memory item, in which session, at which step. This enables cross-agent causal attribution. Without provenance, a failure caused by a memory item written by a different agent in a prior session appears causally disconnected from its actual source - which is an attribution failure that could have real governance consequences.

---

6. Why Existing Approaches Fall Short

Several research efforts address adjacent problems but leave the core gap intact. This paper aims to be precise about where they fall short rather than dismissive.

MAIF (arxiv:2511.15097) applies cryptographic hash chains to agent artifact integrity. It focuses on the artifacts agents produce - outputs, files, generated content - rather than on the causal chain that produced them. It can tell you whether an artifact was tampered with after creation; it cannot tell you why the agent created it in the first place. That's a meaningful distinction.

LLM Workflow Provenance (arxiv:2509.13978) captures runtime metadata for workflow reproducibility. It records what happened with enough fidelity to reproduce a run - but reproducibility and causal attribution are different goals. Knowing you can replay a session doesn't tell you which environmental factor caused the failure within it.

MCP Security Research has produced useful work on provenance metadata for resource retrievals - tracking where data came from before it enters an agent's context. This addresses the

pre-execution problem well. It doesn't address the post-execution attribution problem: given that a failure occurred, which input caused it.

General observability platforms - LangSmith, Weave, and similar - provide structured logging and tracing for agent frameworks. These are valuable and fully complementary to AgentLedger. They surface what the agent did in a readable form. They don't produce tamper-evident records, don't compute causal attribution, and aren't designed to satisfy regulatory audit requirements. They're excellent diagnostics tools, not governance infrastructure.

The shared gap: none of these combines versioned environment state, causal graph construction, deterministic attribution ranking, and tamper-evident governance records into a single auditable primitive. That combination is what AgentLedger proposes.

---

7. Regulatory Applicability

AgentLedger's causal record is designed to answer the accountability questions that current regulations are increasingly demanding.

GDPR Article 22 requires that automated decision-making systems provide meaningful information about the logic involved. A causal record identifying the specific environmental inputs that drove a decision is a direct answer to this requirement - not a general description of how the model works, but a record of what was actually present and active for this specific decision at this specific moment.

SOX requires tamper-evident audit trails for financial processes involving automated systems. AgentLedger's append-only, hash-chained ledger is designed to satisfy this structurally - any alteration to any record is detectable without requiring trust in the record-keeper.

The EU AI Act requires technical documentation for high-risk AI systems demonstrating that the system operated within defined parameters. The Change Envelope is that documentation - a versioned record of every parameter that could influence agent behavior, captured at the time of the action.

NIST AI RMF identifies traceability and explainability as core risk management properties. The Reasoning DAG and causal ranking together constitute a traceability record from environmental inputs to specific actions - which is precisely what the RMF is asking for.

The governance artifact AgentLedger produces is not a general account of how the agent works. It is a specific, replayable, tamper-evident record of why *this* agent took *this* action at *this* moment - which is what regulators, auditors, and incident responders actually need, and what nothing currently provides.

---

8. Open Problems

AgentLedger is a proposed architecture, not a deployed system. This work is explicit about what remains unsolved, because hand-waving over open problems is a disservice to the research community.

Reasoning DAG fidelity. The causal graph used for attribution would be inferred from tool call sequences, context diffs, and action timestamps - signals that already exist in agent frameworks. How faithfully that inference reconstructs the actual reasoning chain is an empirical question that cannot be answered from architecture alone. In constrained, well-instrumented frameworks, fidelity is likely high. In open-ended frameworks it may not be. A standard trace schema emitted natively by agent frameworks would improve this significantly, and can reasonably be treated as a prerequisite for high-confidence attribution.

RAG non-determinism. Hashing retrieval output captures that the context changed, but doesn't make retrieval reproducible. The same query may return different chunks tomorrow even if nothing in the underlying documents changed - vector store updates, embedding model changes, and index drift all affect retrieval. AgentLedger can detect this as a causal candidate; it cannot eliminate the underlying instability.

Per-step envelope overhead. Capturing a full environment snapshot at every consequential action has storage and compute costs that haven't been measured against real production deployments. Per-session baseline capture is low-cost. Per-step capture for high-frequency agents is an open engineering question that needs real numbers.

Cross-agent trust boundaries. When one agent's output becomes another agent's context, whose accountability record governs a downstream failure? AgentLedger's memory provenance model provides a partial answer, but the governance question of cross-organizational agent chains - where agent A is operated by one company and agent B by another - is not resolved by this architecture alone. That's a harder problem, and it will require coordination at an industry or regulatory level.

Causal ranking weight tuning. The ranking function uses equal weights for proximity and exclusivity as a principled default. Whether that default produces accurate attribution across different failure types is an empirical question that requires validation against known-cause failure datasets. Developing such datasets remains an open task for the community.

---

9. Conclusion

The agent accountability gap is real, it's growing, and it's underserved. As agents take on more consequential roles, the inability to answer "why did the agent do that" with anything more rigorous than "the LLM decided" is both an engineering failure and a governance failure.

AgentLedger proposes a concrete architecture for closing that gap: a causal ledger that combines versioned environment state, causal graph construction, deterministic attribution ranking, and tamper-evident records into a single auditable primitive. The cryptographic foundations - CT-style append-only logs, Ethereum-style state fingerprinting - are proven at scale in other domains. The application to agent accountability is new.

This paper is a proposal, not a proof. The open problems in Section 8 are genuine, and their resolution requires empirical work that has not yet been done. But the architecture is coherent, the problem is important, and the gap between what exists today and what governance of agentic AI requires is wide enough that a clear proposal for how to close it is worth putting forward.

References

- Google Certificate Transparency. RFC 6962. B. Laurie, A. Langley, E. Kasper. 2013.
- Ethereum Yellow Paper. G. Wood. 2014. Revised 2023.
- MAIF: A Machine-readable Agent Interchange Format. arxiv:2511.15097.
- LLM Workflow Provenance. arxiv:2509.13978.
- EU Artificial Intelligence Act. Regulation (EU) 2024/1689.
- NIST AI Risk Management Framework. NIST AI 100-1. 2023.
- GDPR Article 22: Automated individual decision-making. Regulation (EU) 2016/679.